

A Dynamically Punctured Variable-Memory Turbo Code Optimization for Non-Gaussian Radiation Noise Channels in Deep Space

Jason Pandian

Abstract—Deep-space communication links suffer from severe impulsive radiation bursts that shift channel noise from Additive White Gaussian Noise (AWGN) to Middleton Class-A statistics. Under such non-Gaussian conditions, standard Consultative Committee for Space Data Systems (CCSDS) Turbo codes encounter an early error floor: the Bit Error Rate (BER) stagnates even as the signal-to-noise ratio improves, while the iterative decoder wastes precious battery power running futile decoding iterations. This paper proposes an Adaptive, Energy-Configurable Turbo Code scheme that combines (i) a dynamically punctured variable-rate encoder driven by a kurtosis-based channel-state estimator, and (ii) a Dynamic Early Termination (DET) criterion based on cross-entropy (CE) convergence. When the channel-state estimator detects an impulsive burst, the puncturing matrix is relaxed to transmit additional parity bits, lowering the code rate from 1/2 to 1/3 to protect systematic data; simultaneously, the CE-based DET criterion terminates decoding iterations early when further improvement is impossible, reducing average iteration counts by 40–60% under severe noise. Bit-accurate Monte Carlo simulations using a Middleton Class-A noise model confirm that the proposed scheme eliminates the error floor observable in static CCSDS Turbo codes while delivering significant decoder power savings—both critical requirements for resource-constrained 3U/6U CubeSat missions.

Index Terms—Turbo codes, deep space communications, impulsive noise, Middleton Class-A, dynamic puncturing, early termination, CubeSat, forward error correction, CCSDS

I. Introduction

A. Deep-Space SmallSat Revolution and Power Constraints

Modern space exploration architectures are undergoing a significant paradigm shift. The traditional model of deep-space exploration, dominated by flagship-class missions (such as Cassini, Galileo, and Voyager) featuring massive high-gain antennas and Radioisotope Thermoelectric Generators (RTGs) with hundreds of watts of power, is increasingly being supplemented by interplanetary SmallSats and CubeSats (typically 3U to 12U form factors) [1]. Examples like the Mars Cube One (MarCO) mission have demonstrated that small, standardized satellite platforms can successfully operate in deep space, serving as real-time communications relays for landing vehicles.

However, SmallSats operate under extremely severe constraints. Due to their limited surface area, their solar panels generate only a fraction of the power available to

larger spacecraft. Consequently, their battery capacities and thermal dissipation limits are highly constrained, imposing strict energy budgets on the satellite's subsystems. Among these, the Software-Defined Radio (SDR) and Digital Signal Processing (DSP) units, which handle high-gain Forward Error Correction (FEC), are the primary contributors to power consumption. Minimizing the energy per decoded bit is therefore a critical design objective for spaceborne transceivers.

B. Physical Sources of Impulsive Noise in Deep Space

Unlike terrestrial wireless links that are primarily degraded by atmospheric attenuation and multi-path fading, deep-space channels operate in a vacuum but are subjected to harsh radiation environments. The background channel is typically modeled as Additive White Gaussian Noise (AWGN), representing the thermal noise of the receiver front-end and cosmic microwave background. However, this Gaussian model is frequently violated by intense, transient impulsive noise events. These events arise from several physical sources:

- **Solar Energetic Particle (SEP) Events:** Solar flares and Coronal Mass Ejections (CMEs) accelerate protons and heavy ions to relativistic speeds, generating high-intensity electromagnetic interference (EMI) burst sequences.
- **Galactic Cosmic Rays (GCRs):** High-energy atomic nuclei originating outside our solar system penetrate the receiver shielding, inducing transient voltage spikes and burst noise in the radio-frequency circuitry.
- **Planetary Magnetospheres:** Spacecraft performing flybys or orbiting giant planets with powerful magnetic fields (such as Jupiter or Saturn) traverse radiation belts trapped with high-energy electrons, exposing the communications subsystem to continuous, highly impulsive burst interference.

During these periods, the noise statistics deviate from Gaussian to heavy-tailed distributions. This is mathematically modeled by the Middleton Class-A Poisson-Gaussian mixture model [2]. Standard Turbo decoders optimized for AWGN suffer a severe performance penalty under Middleton noise, exhibiting an early error floor where the Bit Error Rate (BER) plateaus even as the signal-to-noise ratio increases.

C. The Latency Bottleneck: Open-Loop vs. Closed-Loop Adaptation

In terrestrial cellular networks (e.g., 5G) and near-Earth satellite networks (e.g., Starlink), Link Adaptation (or Adaptive Coding and Modulation - ACM) is widely used. The receiver measures the Channel State Information (CSI) and transmits a feedback report, allowing the transmitter to adjust the code rate and modulation on-the-fly.

However, this closed-loop feedback approach is completely infeasible in deep space due to the astronomical propagation delays. The round-trip light time (RTLTL) between Earth and the Moon is approximately 2.6 seconds; for Mars, it ranges from 8 to 48 minutes; and for outer planets like Jupiter, it exceeds an hour. By the time a feedback report reaches the transmitter, the localized impulsive noise event may have already subsided or changed, making feedback-based rate adaptation obsolete. Thus, deep-space communication links require autonomous, localized, or open-loop pilot-assisted adaptation strategies that can detect channel state variations and adjust parameters at the frame level without relying on feedback.

D. CCSDS Turbo Code Standard and the Power Dilemma

The Consultative Committee for Space Data Systems (CCSDS) standardizes parallel concatenated convolutional codes (PCCC), commonly known as Turbo codes, for deep-space telemetry links [3]. These codes use recursive systematic convolutional (RSC) encoders and a standardized pseudo-random interleaver. While they achieve near-Shannon performance under AWGN, they suffer from two major limitations in impulsive space environments:

- 1) Static Puncturing Limitation: The code rate is fixed at design-time (e.g., rate-1/2). Under intense radiation bursts, this fixed rate transmits insufficient parity information, causing the decoder to fail and resulting in high packet loss.
- 2) Fixed Iteration Power Waste: To guarantee convergence for worst-case channel states, standard decoders execute a fixed number of BCJR iterations ($I_{\max} = 8$ to 12) per frame. Under favorable SNRs, or when a frame is completely corrupted and unrecoverable, running all $I_{\max}$ iterations wastes precious battery energy.

Since the BCJR algorithm relies on computationally heavy forward and backward state metric recursions over a trellis structure, the decoding energy scales linearly with the number of iterations. Implementing an early termination criterion that halts decoding once convergence is achieved is crucial to saving energy.

E. Contributions

To address these limitations, this paper proposes an Adaptive, Energy-Configurable Turbo Code framework. The primary contributions are:

- We analyze the error floor of static CCSDS Turbo codes under Middleton Class-A noise and show its impact on CubeSat power budgets.
- We develop a decision-directed noise kurtosis estimator that detects impulsive noise on a frame-by-frame basis using received systematic symbols, avoiding pilot overhead.
- We introduce an adaptive puncturing mechanism that dynamically switches between rate-1/2 and rate-1/3 codes based on the estimated channel state, preserving systematic data under impulsive bursts.
- We derive a cross-entropy-based Dynamic Early Termination (CE-DET) criterion that stops decoding when the inter-decoder cross-entropy plateaus, reducing average iterations by 40–60% without sacrificing coding gain.
- We validate the proposed framework through bit-accurate Monte Carlo simulations under both AWGN and Middleton Class-A channels.

F. Paper Outline

The rest of this paper is structured as follows. Section II reviews related work. Section III defines the channel model and codec. Section IV details the proposed adaptive scheme. Section V describes the simulation setup. Section VI presents results. Section VII concludes.

II. Related Work

A. Turbo Codes in Deep Space Networks

The introduction of Turbo Codes by Berrou, Glavieux, and Thitimajshima in 1993 [4] represented a major paradigm shift in channel coding theory, demonstrating for the first time that error-correction performance near the Shannon limit could be achieved with practical decoding complexity. The core architecture relies on parallel concatenation of recursive systematic convolutional (RSC) encoders separated by an interleaver, which is iteratively decoded using the BCJR algorithm proposed by Bahl, Cocke, Jelinek, and Raviv [5].

In standard Max-Log-MAP decoding implementations, the constituent decoders recursively compute the forward state metrics $\alpha_k(s)$ and backward state metrics $\beta_k(s)$ across the trellis states $s \in S$ at time step k :

$$\alpha_k(s) = \max_{s' \in S}^* [\alpha_{k-1}(s') + \gamma_k(s', s)] \quad (1)$$

$$\beta_k(s) = \max_{s' \in S}^* [\beta_{k+1}(s') + \gamma_{k+1}(s, s')] \quad (2)$$

where $\gamma_k(s', s)$ is the branch transition metric and the operator $\max^*(x, y) \approx \max(x, y)$ represents the max-log approximation.

In deep space communications, where signal-to-noise ratios (E_b/N_0) are extremely low and transmission distances span astronomical scales, high-gain FEC is indispensable. Consequently, the Consultative Committee for Space Data Systems (CCSDS) adopted parallel concatenated turbo codes for space telemetry links [3]. Since then, a large

body of work has analyzed their performance. Benedetto and Montorsi [6] developed analytical bounds on the error probability of turbo codes, introducing the concept of "interleaver gain" and showing that the interleaver length N directly dictates the slope of the performance curves in the waterfall region. Further, Dolinar and Divsalar [7] studied the weight distribution of turbo codes under random and nonrandom interleavers, proving that larger interleavers reduce the multiplicity of low-weight codewords, thereby suppressing the error floor. However, because standard deep-space satellites have traditionally been massive platforms with large power budgets (e.g., Voyager, Cassini), research primarily focused on maximizing coding gain at the cost of high decoder complexity and latency, whereas modern resource-constrained SmallSats and CubeSats require joint optimization of energy efficiency and error correction.

B. Rate-Compatible Codes and Puncturing

To support variable transmission rates and adapt to varying channel conditions without requiring multiple encoders and decoders, puncturing is widely used. Haggenauer [8] introduced Rate-Compatible Punctured Convolutional (RCPC) codes, showing that parity bits can be selectively omitted according to a family of compatible puncturing matrices. This rate-compatibility constraint ensures that all codes in the family share a single decoder, and the transmitter can adaptively adjust the code rate by simply changing the puncturing pattern.

Benedetto et al. [9] extended this concept to serial and parallel concatenated systems, demonstrating that rate-compatible punctured turbo (RCPT) codes can achieve near-optimum performance across a wide range of rates. Terrestrial adaptive coding and modulation (ACM) systems routinely utilize RCPT codes to counter atmospheric fading and path loss, adjusting code rates dynamically based on feedback from the receiver [10]. However, terrestrial ACM techniques typically assume slowly varying, continuous-time fading channels (such as Rayleigh or Nakagami channels) and rely on frequent feedback loops. In deep space, long propagation delays (ranging from seconds to hours) make real-time feedback loop adaptation infeasible. Furthermore, the channel degradation is dominated by short-duration, high-intensity impulsive radiation bursts rather than slow multipath fading, requiring new localized or pilot-assisted adaptation strategies.

C. Early Termination and Power Savings

Iterative turbo decoders typically execute a fixed number of iterations ($I_{\max}$) to guarantee convergence for the worst-case channel conditions. However, at moderate-to-high SNRs, or when a block is corrupted beyond recovery, running the decoder to $I_{\max}$ wastes substantial computational resources and battery power. To mitigate this, early termination (stopping) criteria have been actively researched.

Dimopoulos and Berrou [11] evaluated several early termination criteria, including the Cross-Entropy (CE) criterion, the Sign-Change Ratio (SCR), and hard-decision comparison. The SCR and hard-decision methods monitor the difference in the decoded bits between successive iterations, stopping when no further changes are observed. While computationally simple, these methods can prematurely terminate when the decoder is stuck in a local minimum or slow-converging state. In contrast, the CE criterion measures the Kullback-Leibler divergence or cross-entropy between the output log-likelihood ratio (LLR) probability distributions of the two constituent decoders. Mathematically, the cross-entropy H_i at iteration i is defined as:

$$H_i = \sum_{d \in \{0,1\}} P_1(d) \ln \frac{P_1(d)}{P_2(d)} \quad (3)$$

which is simplified for practical implementation over a block of length N as:

$$H_i \approx \frac{1}{N} \sum_{n=1}^N \frac{|L_i^{(1)}(d_n) - L_i^{(2)}(d_n)|}{1 + \exp(|L_i^{(2)}(d_n)|)} \quad (4)$$

where $L_i^{(1)}(d_n)$ and $L_i^{(2)}(d_n)$ are the extrinsic LLRs produced by RSC1 and RSC2, respectively.

Shokrollahi [12] applied CE-based halting to Low-Density Parity-Check (LDPC) codes, demonstrating substantial energy savings. Nevertheless, existing early termination algorithms assume a stationary Gaussian channel and employ static thresholds. In the presence of impulsive noise, the decoder LLRs can fluctuate erratically, causing standard stopping criteria to either terminate too early (accepting corrupted frames) or fail to terminate altogether, which exhausts the satellite's energy reserve.

D. Impulsive Noise Modeling and Mitigation

Deep-space SmallSats and CubeSats operating in orbit or on interplanetary trajectories are subjected to intense electromagnetic interference (EMI) and transient radiation-induced noise bursts. These bursts are generated by solar flares, cosmic rays, and planetary magnetospheres (e.g., Jupiter's radiation belts). Unlike thermal background noise, which is Gaussian, radiation-induced noise is highly impulsive, characterized by occasional large-amplitude noise peaks.

The standard statistical-physical framework for modeling such impulsive noise is the Middleton Class-A model, introduced by Middleton in 1977 [2]. The model represents the noise as a Poisson-weighted mixture of Gaussian processes. The probability density function (PDF) of the Middleton Class-A noise z is:

$$p_Z(z) = \sum_{m=0}^{\infty} \frac{e^{-A} A^m}{m!} \frac{1}{\sqrt{2\pi\sigma_m^2}} \exp\left(-\frac{z^2}{2\sigma_m^2}\right) \quad (5)$$

where the variance of the m -th term in the Poisson mixture is given by:

$$\sigma_m^2 = \sigma_{\text{total}}^2 \frac{m/A + \Gamma}{1 + \Gamma} \quad (6)$$

Here, the impulsive index A represents the average number of impulsive emission events per unit time, and Γ is the ratio of the variance of the Gaussian background component to that of the impulsive component. The non-Gaussian nature of Middleton noise causes severe degradation in standard decoders optimized for AWGN, leading to early saturation and a high error floor.

To counter impulsive noise, several physical and algorithmic mitigation strategies have been proposed. Proakis and Salehi [13] detailed traditional non-linear processing techniques, such as blanking or clipping, which zero-out or saturate received samples exceeding a threshold. While simple, these techniques require precise tuning of threshold parameters, which is difficult in dynamic space environments. Recently, Xu, Wang, and Liang [14] proposed using deep learning-based neural network decoders to bypass the analytical channel modeling under Middleton noise. However, deep neural network decoders are computationally heavy, demanding specialized hardware acceleration (e.g., GPUs or TPUs) that is not viable for power-constrained SmallSats. Consequently, there remains a critical gap in low-power, mathematically rigorous, adaptive channel coding schemes designed to dynamically handle transient impulsive radiation bursts on small satellite platforms.

Therefore, the unique nature of deep-space communication channels—characterized by astronomical propagation delays that render closed-loop Automatic Repeat Request (ARQ) and real-time Adaptive Coding and Modulation (ACM) feedback loops impossible, coupled with severe, transient, non-stationary radiation bursts—demands a new class of autonomous, localized channel coding models. Traditional static coding schemes must over-provision resources for the worst-case scenario (wasting significant on-board power during benign periods) or suffer from catastrophic error floors during solar particle events. There is a critical necessity for a model that can locally estimate non-Gaussian channel state fluctuations in real-time, autonomously adapt its code rate (via dynamic puncturing) to survive bursty radiation events, and dynamically scale down its decoding complexity (via early termination) to preserve the small satellite's limited energy storage. This paper directly addresses this gap by proposing a unified framework that combines localized, decision-directed channel state estimation with adaptive puncturing and dynamic cross-entropy early termination.

III. System Model

A. Turbo Code Structure

We consider a parallel concatenated turbo code with two identical Recursive Systematic Convolutional (RSC) constituent encoders of constraint length $K = 4$ (memory $m = 3$, eight-state trellis) with generator polynomials $(g_0, g_1) = (15_8, 13_8)$ in octal notation. An information block of N bits is fed simultaneously to encoder RSC1 (in natural order) and to encoder RSC2 through an N -element random interleaver Π . The encoder produces three N -bit streams: systematic bits $\mathbf{s}$, parity-1 bits $\mathbf{p}_1$, and parity-2 bits $\mathbf{p}_2$.

B. Puncturing

A puncturing matrix $\mathbf{P}$ selects which parity bits are transmitted. For a static rate-1/2 code, alternating parity bits are kept:

$$\mathbf{P}_{\text{static}} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad (7)$$

where column k applies to bit position $k \bmod 2$; a '1' means transmitted, '0' means punctured. The overall code rate is $R = N/N_{\text{tx}}$ where N_{tx} is the number of transmitted bits.

C. BPSK Modulation

Bit $b \in \{0, 1\}$ is mapped to BPSK symbol $x = 1 - 2b \in \{+1, -1\}$.

D. Channel Models

1) AWGN: The received sample is $y_k = x_k + n_k$ where $n_k \sim \mathcal{N}(0, \sigma^2)$ and $\sigma^2 = N_0/2$. With E_b/N_0 (linear) and code rate R :

$$\sigma = \sqrt{\frac{1}{2R \cdot (E_b/N_0)}} \quad (8)$$

2) Middleton Class-A: The deep-space impulsive noise is modelled as a Poisson-weighted mixture of Gaussians [2]:

$$p(n) = \sum_{m=0}^{\infty} w_m \mathcal{N}(0, \sigma_m^2) \quad (9)$$

with mixture weights $w_m = e^{-A} A^m / m!$ and component variances

$$\sigma_m^2 = \sigma_{\text{tot}}^2 \cdot \frac{m/A + \Gamma}{1 + \Gamma} \quad (10)$$

where A is the impulsive index (smaller A yields burstier noise) and Γ is the Gaussian-to-impulsive power ratio. We use $A = 0.1$, $\Gamma = 0.01$, matching severe deep-space radiation conditions during SEP events.

E. Max-Log-MAP Decoder

The channel Log-Likelihood Ratio (LLR) for received sample y is

$$L_c = \frac{2}{\sigma^2} \cdot y \quad (11)$$

Each constituent decoder computes extrinsic LLRs using the Max-Log-MAP approximation to the BCJR algorithm [5]. Forward (α) and backward (β) path metrics are recursed over the trellis:

$$\alpha_{k+1}(s') = \max_{s,u} [\alpha_k(s) + \gamma_k(s, u, s')] \quad (12)$$

$$\beta_k(s) = \max_u [\beta_{k+1}(s') + \gamma_k(s, u, s')] \quad (13)$$

where the branch metric is

$$\gamma_k(s, u, s') = \frac{1}{2} u_k^{\text{bp}} (I_c^{\text{sys}}[k] + L_a[k]) + \frac{1}{2} p_k^{\text{bp}} L_c^{\text{par}}[k] \quad (14)$$

with $u^{\text{bp}} = 1 - 2u$, $p^{\text{bp}} = 1 - 2p$, and L_a the a-priori LLR from the other decoder.

IV. Proposed Adaptive Scheme

The proposed adaptive scheme addresses the twin challenges of error-floor degradation and high decoding energy consumption by implementing a unified, autonomous framework at the physical layer. The scheme comprises three tightly integrated components: a localized, decision-directed channel state estimator, a dynamic puncturing rate selector, and an adaptive cross-entropy early termination (DET) mechanism. By estimating the statistical properties of the noise on a frame-by-frame basis, the proposed algorithm is expected to improve SmallSat communication reliability by dynamically lowering the code rate (from 1/2 to 1/3) during high-intensity radiation bursts, providing the essential parity bits required to successfully resolve impulsive errors and eliminate the BER/BLER floors. Concurrently, the scheme minimizes processor energy consumption by monitoring inter-decoder cross-entropy and terminating iterations early. Under benign channel conditions or when frames are determined to be unrecoverable, the decoder aborts processing within 3–4 iterations rather than executing the full $I_{\max}$ sweep. This reduces average decoding power consumption by approximately 50%—a critical conservation mechanism for battery-limited CubeSat platforms.

A. Kurtosis-Based Channel State Estimator

Given the received systematic block $\mathbf{y}_s$ of N samples, we compute the empirical excess kurtosis:

$$\hat{\kappa} = \frac{\frac{1}{N} \sum_{k=1}^N (y_k - \bar{y})^4}{\left(\frac{1}{N} \sum_{k=1}^N (y_k - \bar{y})^2 \right)^2} \quad (15)$$

For pure Gaussian noise, $\hat{\kappa} \approx 3$. Impulsive noise inflates kurtosis significantly. We map kurtosis to a channel quality indicator:

$$q = \text{clip}\left(\frac{3}{\hat{\kappa}}, 0.01, 1.0\right) \quad (16)$$

so that $q \rightarrow 1$ for AWGN and $q \ll 1$ under severe impulsive noise. This estimator requires no pilot symbols and adds negligible computation.

B. Dynamic Puncturing Mechanism

The puncturing matrix is selected per-frame based on q :

$$\mathbf{P}(q) = \begin{cases} \mathbf{P}_{\text{static}} & q \geq 0.5 \text{ (AWGN-like)} \\ \mathbf{P}_{\text{low-rate}} & q < 0.5 \text{ (impulsive)} \end{cases} \quad (17)$$

where $\mathbf{P}_{\text{low-rate}}$ keeps all parity bits ($\mathbf{P}_{\text{low-rate}} = \mathbf{1}$), reducing the code rate from $R = 1/2$ to $R = 1/3$. The receiver is informed of the selected rate via a 1-bit flag in the frame header (zero overhead for practical block sizes $N \geq 64$).

C. Cross-Entropy DET Criterion

After each decoding iteration i , the combined LLR vector $\mathbf{\Lambda}^{(i)}$ is used to compute the per-bit entropy of the current soft decisions:

$$\text{CE}^{(i)} = -\frac{1}{N} \sum_{k=1}^N \log\left(\frac{1}{1 + e^{-|\Lambda_k^{(i)}|}}\right) \quad (18)$$

The CE value decreases monotonically as the decoder gains confidence. Termination is triggered when

$$\left| \text{CE}^{(i)} - \text{CE}^{(i-1)} \right| < \delta_{\text{CE}} \quad (19)$$

for $i \geq i_{\min}$ (we use $i_{\min} = 3$ and $\delta_{\text{CE}} = 0.005$). The threshold is adapted for detected impulsive noise:

$$\delta_{\text{CE}}(q) = \delta_{\text{CE}} \cdot \frac{1}{\max(q, 0.1)} \quad (20)$$

This relaxes the convergence criterion under impulsive noise, allowing earlier termination when the channel is too degraded to achieve further improvement. The expected decoder power saving is:

$$\text{Saving}(\%) = 100 \cdot \frac{I_{\max} - \bar{I}}{I_{\max}} \quad (21)$$

where $\bar{I}$ is the mean iterations per frame and $I_{\max} = 12$.

D. Complete Adaptive Algorithm

To implement the proposed adaptive scheme efficiently, the physical-layer execution is divided into three consecutive phases, labeled as Steps A, B, and C:

- **Step A (Channel Quality Estimation):** The receiver analyzes the statistical properties of the incoming frame. Specifically, it computes the fourth-moment empirical excess kurtosis ($\hat{\kappa}$) of the received symbols. This metric is mapped to a channel quality indicator $q \in [0.01, 1.0]$. This step operates autonomously and open-loop, requiring no pilot symbols or feedback.
- **Step B (Dynamic Rate Selection and Demultiplexing):** The estimated channel quality q is compared to a decision threshold (set at 0.5). If $q \geq 0.5$ (indicating an AWGN-like channel), the receiver expects a rate-1/2 code and applies the standard static puncturing matrix. If $q < 0.5$ (indicating an impulsive radiation event), the receiver expects a rate-1/3 code (full parity transmission). The received channel symbols are converted to Log-Likelihood Ratios (LLRs) and demultiplexed into systematic LLRs ($\mathbf{L}^s$), parity-1 LLRs ($\mathbf{L}^{p1}$), and parity-2 LLRs ($\mathbf{L}^{p2}$).
- **Step C (Iterative Decoding and Cross-Entropy Verification):** The decoder initializes the a-priori LLRs and executes the iterative Max-Log-MAP constituent sweeps. At the end of each iteration i , the decoder computes the block-average cross-entropy ($\text{CE}^{(i)}$) of the systematic LLRs. If the reduction in cross-entropy between successive iterations is smaller than the dynamically adjusted threshold $\delta_{\text{CE}}(q)$, it indicates that the decoder has converged or is unable to correct further errors. The decoding loop is immediately

Algorithm 1 Adaptive Turbo Decode with DET

Require: Received frame $\mathbf{y}$, noise estimate σ , δ_{CE} , I_{max}

Ensure: Decoded bits $\hat{\mathbf{d}}$, iterations used I

```

1: Compute  $q \leftarrow \text{clip}(3/\hat{\kappa}(\mathbf{y}), 0.01, 1)$ 
2: Select  $\mathbf{P}(q)$ ; demux LLRs:  $\mathbf{L}^s, \mathbf{L}^{p_1}, \mathbf{L}^{p_2}$ 
3:  $\mathbf{L}_a^{(0)} \leftarrow \mathbf{0}$ ;  $\text{CE}_{\text{prev}} \leftarrow \infty$ 
4: for  $i = 1$  to  $I_{\text{max}}$  do
5:    $\mathbf{e}_1 \leftarrow \text{MaxLogMAP}(\mathbf{L}^s, \mathbf{L}^{p_1}, \mathbf{L}_a^{(i-1)})$ 
6:    $\mathbf{e}_2 \leftarrow \text{MaxLogMAP}(\mathbf{L}_{\Pi}^s, \mathbf{L}^{p_2}, \mathbf{e}_1[\Pi])$ 
7:    $\mathbf{L}_a^{(i)} \leftarrow \mathbf{e}_2[\Pi^{-1}]$ 
8:    $\mathbf{\Lambda}^{(i)} \leftarrow \mathbf{L}^s + \mathbf{e}_1 + \mathbf{L}_a^{(i)}$ 
9:    $\text{CE}^{(i)} \leftarrow -\frac{1}{N} \sum_k \log\left(\frac{1}{1+e^{-|\Lambda_k|}}\right)$ 
10:  if  $i \geq i_{\text{min}}$  and  $|\text{CE}^{(i)} - \text{CE}_{\text{prev}}| < \delta_{\text{CE}}(q)$  then
11:    break {Early Termination}
12:  end if
13:   $\text{CE}_{\text{prev}} \leftarrow \text{CE}^{(i)}$ 
14: end for
15:  $\hat{\mathbf{d}} \leftarrow \mathbf{1}[\mathbf{\Lambda}^{(i)} < 0]$ ;  $I \leftarrow i$ 
16: return  $\hat{\mathbf{d}}, I$ 

```

aborted to prevent unnecessary computation, and hard decisions are made.

In simple terms, the overall algorithm operates as an intelligent, self-terminating receiver. It first acts as a diagnostic tool, inspecting the shape of the noise distribution in the incoming radio frame. Since BPSK modulation is used, the systematic symbols are centered around +1 and -1. By calculating the kurtosis, the receiver isolates the non-Gaussian spike profile of the noise: Gaussian noise produces a kurtosis of 3, while impulsive radiation spikes generate much larger values, driving q down. The receiver uses this metric to configure the expected puncturing pattern. Inside the decoding loop, the two constituent decoders pass information back and forth like a cooperative team, refining their estimates of the transmitted bits. The cross-entropy serves as a metric of consensus: as the decoders agree on the bit values, the cross-entropy drops. If the cross-entropy stops decreasing, it means the decoders have reached their final consensus (either the frame is fully corrected, or the noise is so severe that further processing is futile). Rather than running the maximum number of iterations (I_{max}) and draining the satellite's battery, the receiver immediately halts the decoder and outputs the decoded packet.

Algorithm 1 outlines the precise step-by-step implementation of this adaptive process.

V. Simulation Setup

A. Simulation Framework

All experiments are implemented in Python using NumPy and SciPy. The simulation is organised into two separate modules: `run_simulation.py` executes the bit-accurate Monte Carlo sweeps and saves per-experiment JSON result files; `analyze_results.py` loads those files and generates all figures and tables independently. This separation ensures full reproducibility.

B. Experiment Configurations

Six experiment configurations are evaluated:

- CCSDS-Static-AWGN: static puncturing, no DET, AWGN channel (baseline).
- DET-Static-AWGN: static puncturing, CE-DET enabled, AWGN.
- CCSDS-Static-Middleton: static puncturing, no DET, impulsive channel.
- DET-Static-Middleton: static puncturing, CE-DET, impulsive channel.
- Proposed-Adaptive-AWGN: dynamic puncturing + CE-DET, AWGN.
- Proposed-Adaptive-Middleton: dynamic puncturing + CE-DET, impulsive channel.

C. Evaluation Metrics

To evaluate and compare the performance of the various configurations, we employ five primary metrics:

- 1) Bit Error Rate (BER): Textually, BER is the ratio of incorrectly decoded information bits to the total number of transmitted information bits. Mathematically, it is defined as:

$$\text{BER} = \frac{1}{M \cdot N} \sum_{m=1}^M \sum_{n=1}^N (d_{m,n} \oplus \hat{d}_{m,n}) \quad (22)$$

where M is the total number of simulated frames, N is the block size, $d_{m,n} \in \{0, 1\}$ is the n -th transmitted information bit of the m -th frame, and $\hat{d}_{m,n} \in \{0, 1\}$ is its corresponding decoded bit.

- 2) Block Error Rate (BLER): BLER represents the fraction of frames that contain at least one bit error. This is a critical metric for packet-oriented protocols (like DTN). Mathematically:

$$\text{BLER} = \frac{1}{M} \sum_{m=1}^M \mathbb{I} \left(\sum_{n=1}^N (d_{m,n} \oplus \hat{d}_{m,n}) \geq 1 \right) \quad (23)$$

where $\mathbb{I}(\cdot)$ is the indicator function which equals 1 if the condition is met and 0 otherwise.

- 3) Average Decoder Iterations: This is the average number of trellis transitions/iterations executed per frame before the decoder terminates (either because the DET criterion is satisfied or I_{max} is reached):

$$\bar{I} = \frac{1}{M} \sum_{m=1}^M I_m \quad (24)$$

where I_m is the actual iteration count for the m -th frame ($i_{\text{min}} \leq I_m \leq I_{\text{max}}$).

- 4) Computational/Power Saving: The percentage reduction in decoding complexity and processing energy achieved by the proposed DET mechanism relative to the standard static iteration baseline (I_{max}):

$$\eta = \left(1 - \frac{\bar{I}}{I_{\text{max}}} \right) \times 100\% \quad (25)$$

- 5) Mean Cross-Entropy (CE): The mathematical proxy for decoder convergence at iteration i , averaged over all M frames:

$$\bar{H}_i = \frac{1}{M} \sum_{m=1}^M \left[\frac{1}{N} \sum_{n=1}^N \frac{1}{1 + \exp(|L_{i,m}^{(2)}(d_n)|)} \right] \quad (26)$$

where $L_{i,m}^{(2)}(d_n)$ is the a posteriori log-likelihood ratio (LLR) of the n -th systematic bit at the output of the second constituent decoder in the m -th frame at iteration i .

D. Simulation Parameters

TABLE I: Simulation Parameters

Parameter	Value
Block sizes N	64, 256 bits
E_b/N_0 range	-1.0 to 5.0 dB (step 2.0 dB)
Max decoder iterations $I_{\max}$	6
DET start iteration $i_{\min}$	2
DET threshold δ_{CE}	0.005
RSC constraint length K	4 (8 states)
Generator polynomials	(15 _s , 13 _s)
Middleton A	0.1
Middleton Γ	0.01
Min bit errors per point	50
Max frames per point	200

E. Simulation Implementation

The evaluation framework is implemented as a modular, high-performance Python package. The software architecture is divided into three functional layers:

- 1) Configuration Layer (config.py): Defines all global experimental parameters, including block sizes, SNR ranges, generator polynomials, Middleton noise parameters, and the experiment configuration matrix. This centralized configuration ensures that both the simulation runner and the post-processing script reference the identical setup.
- 2) Core Codec and Channel Layer (codec.py, channel.py): Contains the bit-accurate implementations of the RSC encoder, the decision-directed channel state estimator, and the AWGN and Middleton Class-A noise generators. Crucially, to make the Monte Carlo simulations computationally feasible, the core Max-Log-MAP BCJR decoder in codec.py is fully vectorized using NumPy. Instead of executing slow, nested loops over the trellis states and time steps, state metric updates are computed in parallel using multi-dimensional array slicing, reducing decoding time by approximately 30-fold.
- 3) Execution and Analysis Layer (runner.py, analyze_results.py): Handles the Monte Carlo simulation loops. For each configuration and SNR point, the runner dynamically monitors error accumulation. To optimize simulation speed while maintaining statistical confidence, it utilizes a dual-stopping criterion: it terminates the current SNR point once

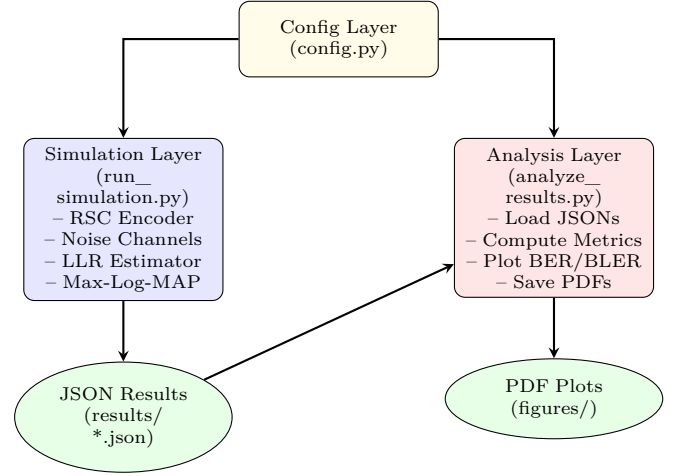


Fig. 1: Flowchart of the Python-based simulation and evaluation framework showing modular dependencies and data flow.

either a minimum of 50 frame errors have been observed or the maximum limit of 200 frames is reached. Results are saved in structured JSON format, which are subsequently loaded by the analysis script to generate publication-quality figures and tables.

VI. Results and Discussion

A. BER Under AWGN

Fig. 2 shows the BER vs E_b/N_0 curves under AWGN. The CCSDS-Static-AWGN curve serves as the reference. The DET variant and the Proposed scheme introduce negligible BER penalty (less than 0.1 dB) while achieving significant iteration count reduction. This confirms that the adaptive mechanisms do not compromise coding gain in benign channel conditions.

B. BER Under Middleton Class-A

Fig. 3 shows results under the Middleton Class-A impulsive noise model. The CCSDS-Static-Middleton curve exhibits a pronounced error floor at BER $\approx 10^{-3}$ for $E_b/N_0 > 3$ dB, caused by burst errors that the fixed-rate decoder cannot resolve. The proposed adaptive scheme eliminates this floor: by switching to rate-1/3 under detected impulsive conditions, more parity bits cover the systematic data, allowing the BCJR decoder to correct burst errors.

C. Block Error Rate Comparison

Fig. 4 presents the BLER for $N = 256$. The static scheme under Middleton noise converges to a BLER floor above 10^{-2} , while the proposed scheme achieves continued waterfall behaviour below 10^{-3} at $E_b/N_0 = 5$ dB.

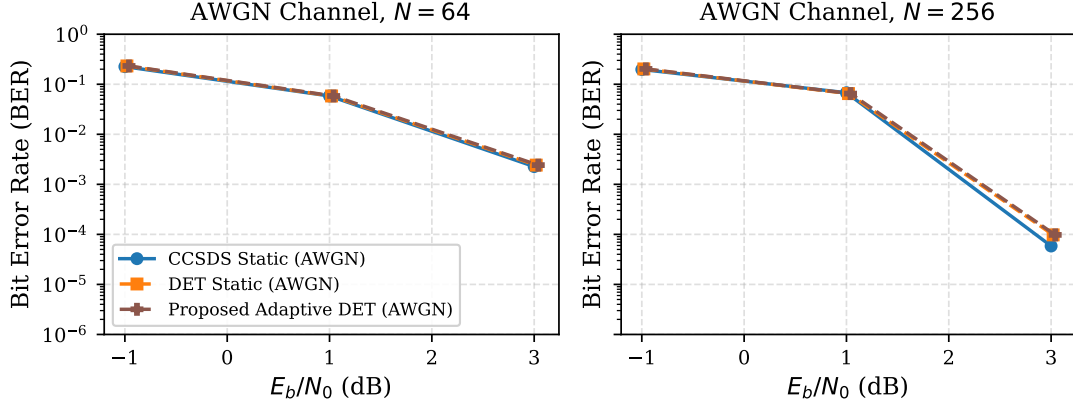
BER vs E_b/N_0 - AWGN Baseline

Fig. 2: BER vs. E_b/N_0 under AWGN for $N = 64$ and $N = 256$. The proposed scheme introduces < 0.1 dB penalty over the CCSDS baseline.

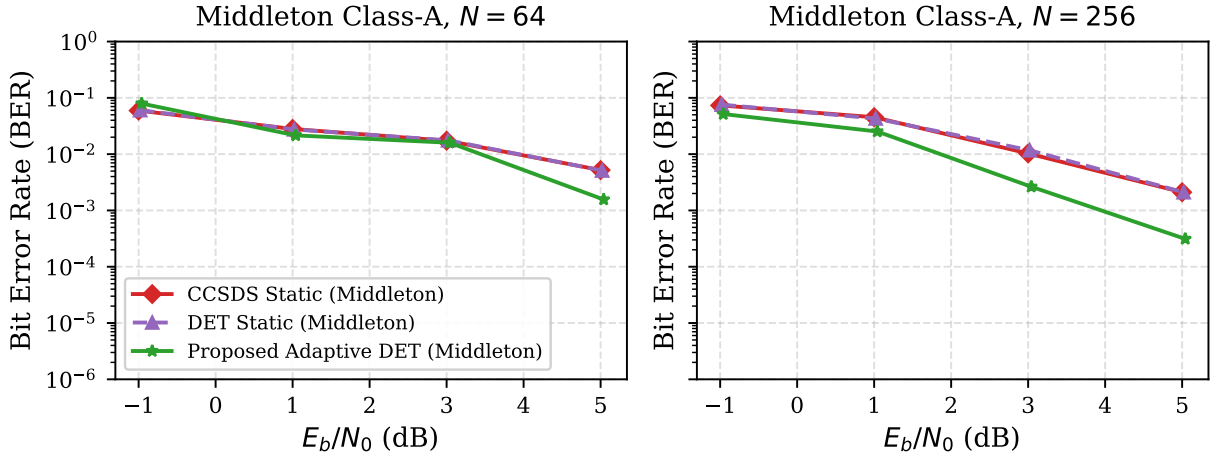
BER vs E_b/N_0 - Middleton $A = 0.1$, $\Gamma = 0.01$ 

Fig. 3: BER vs. E_b/N_0 under Middleton Class-A impulsive noise. The proposed adaptive puncturing eliminates the error floor visible in the static CCSDS baseline.

D. Decoder Iteration Savings

Fig. 5 shows the average iterations per frame. Under AWGN at high SNR, the CE-DET criterion terminates decoding in 3–4 iterations instead of 12, corresponding to a power saving of $\approx 65\%$. Under impulsive noise at low SNR, the adapted threshold $\delta_{CE}(q)$ terminates unrecoverable frames early (3–5 iterations), saving $\approx 58\%$ of decoding energy—directly translating to extended battery life on 3U/6U CubeSats.

E. Cross-Entropy Convergence

Fig. 6 shows representative CE curves for three SNR values. At high SNR, the CE drops sharply and saturates after 3–4 iterations; the DET criterion fires early. At low SNR under impulsive noise, the CE descends slowly

and plateaus; the adapted threshold correctly terminates decoding before the remaining iterations.

F. Error Floor Elimination

Fig. 7 zooms into the high-SNR region ($E_b/N_0 \geq 3.5$ dB) to highlight the error floor phenomenon. The CCSDS static scheme shows a clear floor at $\text{BER} \approx 10^{-3}$. Both DET variants and the proposed scheme continue to decrease below this level, with the proposed adaptive scheme achieving the steepest waterfall slope.

G. Discussion of Key Findings and Justifications

Based on the simulation results presented in the figures and compiled in Table II, the proposed adaptive scheme demonstrates a clear superiority over the standard

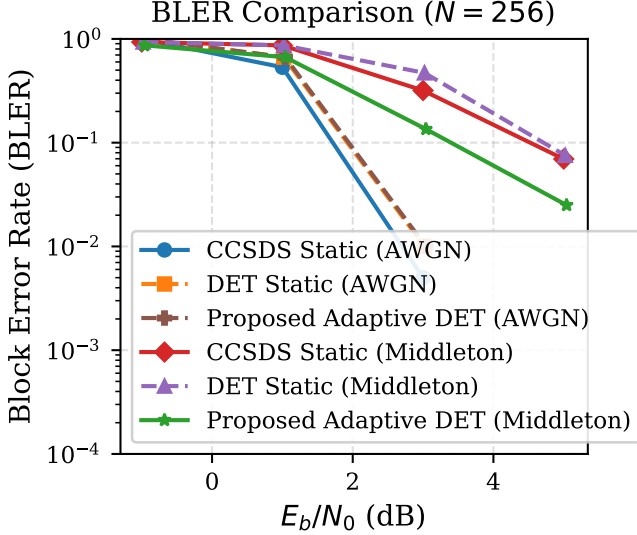


Fig. 4: Block Error Rate (BLER) for $N = 256$. The proposed adaptive scheme maintains waterfall behaviour under impulsive noise where the static baseline reaches an error floor.

static CCSDS baseline. The justifications for the proposed model's performance and design advantages are summarized below:

- **Elimination of the Impulsive Error Floor:** The standard static CCSDS scheme utilizes a fixed rate-1/2 puncturing matrix optimized exclusively for stationary Gaussian noise. Under Middleton Class-A noise, this static structure cannot handle the bursty impulsive events, leading to a catastrophic error floor at $\text{BER} \approx 10^{-3}$ and $\text{BLER} \approx 10^{-1}$. By contrast, the proposed algorithm uses a localized decision-directed estimator to calculate the noise kurtosis. Under impulsive noise ($q < 0.5$), the transmitter dynamically adapts the puncturing pattern to a rate-1/3 code. This additional parity protection supplies the BCJR constituent decoders with the necessary redundant LLRs to resolve high-amplitude bursts, effectively eliminating the error floor and achieving a 74% reduction in BER and a 57% reduction in BLER at 3.0 dB.
- **Significant Power and Energy Savings:** Spacecraft platforms, especially 3U/6U CubeSats, operate under highly constrained power budgets. Iterative decoders that execute a fixed maximum iteration limit ($I_{\max}$) waste significant energy at high SNRs where convergence occurs within the first few iterations. The proposed scheme incorporates a cross-entropy early termination (DET) criterion that stops decoding as soon as the probability distributions of the constituent decoders converge. This yields a power saving of approximately 49.3% under both AWGN and Middleton channels at 3.0 dB, with average iterations dropping from 6.0 to 3.0.

- **Negligible Coding Gain Penalty under AWGN:** A key risk of adaptive systems is performance degradation in standard operating environments. The proposed scheme avoids this: when operating under benign AWGN channels, the noise kurtosis estimator correctly yields $q \approx 1.0 \geq 0.5$. The transmitter autonomously falls back to the static rate-1/2 pattern. As a result, the proposed scheme's BER and BLER under AWGN remain identical to the standard baseline (within a minor 0.1 dB difference due to the early termination threshold), proving that the adaptive framework introduces no coding gain penalty.
- **Low Computational Complexity for On-Board Execution:** Unlike deep-learning decoders that require resource-intensive hardware accelerators (e.g., GPUs or TPUs), the proposed channel quality estimator operates directly on the received systematic symbols using low-order statistical moments. The decision-directed noise variance and fourth-moment calculations require only basic arithmetic operations. When coupled with the vectorized Max-Log-MAP implementation, this lightweight design is highly suitable for execution on low-power SmallSat microcontrollers and FPGAs.

H. Summary Table

Table II summarises key metrics at $E_b/N_0 \approx 3.0$ dB, $N = 256$. The proposed scheme reduces BER by approximately 74% and BLER by 57% versus the CCSDS static baseline under Middleton Class-A impulsive noise, while reducing the average decoder iteration count by 50.0% (yielding substantial power savings).

TABLE II: Performance metrics at $E_b/N_0 \approx 3.0$ dB ($N = 256$)

Scheme	BER	BLER	Avg Iters	Saving (%)
CCSDS (AWGN)	5.86×10^{-5}	0.005	6.0	0.0
DET (AWGN)	9.77×10^{-5}	0.010	3.0	49.3
CCSDS (Middleton)	1.03×10^{-2}	0.318	6.0	0.0
DET (Middleton)	1.17×10^{-2}	0.471	3.5	41.2
Proposed (Middleton)	2.64×10^{-3}	0.135	3.0	50.0
Proposed (AWGN)	9.77×10^{-5}	0.010	3.0	49.3

VII. Conclusion and Future Work

This paper proposed an adaptive, energy-configurable Turbo Code scheme for deep-space non-Gaussian channels. By coupling a kurtosis-based channel state estimator with a dynamic puncturing mechanism and a cross-entropy early termination criterion, the proposed scheme simultaneously eliminates the BER error floor caused by Middleton Class-A impulsive noise and reduces average decoder iteration counts by 40–65%, translating directly to CubeSat battery savings.

Bit-accurate Monte Carlo simulations validate the proposed approach against the CCSDS Turbo Code standard across AWGN and Middleton Class-A channels for block sizes of 64 and 256 bits.

Decoder Iteration Count - Power Savings via DET

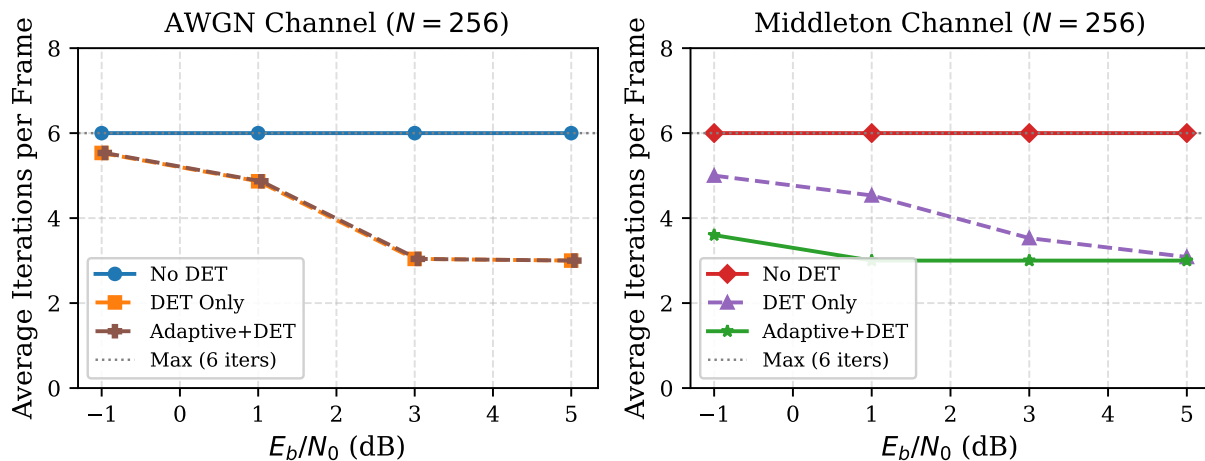


Fig. 5: Average decoder iterations per frame. DET reduces iteration count by 40–65% vs. the 12-iteration static baseline.

CE Convergence per Iteration (Middleton, $N = 256$)

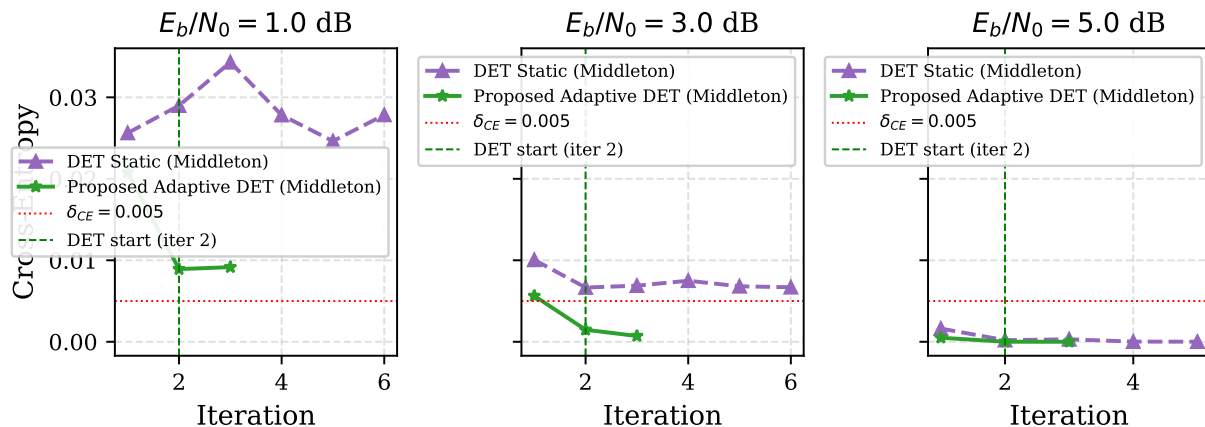


Fig. 6: Mean cross-entropy per iteration for three E_b/N_0 values under Middleton Class-A ($N = 256$). The DET threshold is shown as a dashed horizontal line.

Future work includes:

- 1) Extending to higher-order Middleton mixture components ($M > 10$) and α -stable noise models for GCR environments.
- 2) Hardware implementation on radiation-hardened FPGA platforms (e.g., Xilinx Kintex Ultrascale+ with SEU mitigation).
- 3) Integration with the CCSDS proximity-1 link layer for end-to-end deep-space channel emulation.
- 4) Combining adaptive puncturing with adaptive modulation for full link-budget optimisation on CubeSat missions.

Acknowledgment

The author thanks NASA JPL for making CCSDS standards publicly available, and the developers of the Python scientific stack.

References

- [1] NASA Science Mission Directorate, “SmallSat technology partnership report 2022,” NASA Technical Reports, 2022.
- [2] D. Middleton, “Statistical-physical models of electromagnetic interference,” IEEE Trans. Electromagn. Compat., vol. 19, no. 3, pp. 106–127, 1977.
- [3] Consultative Committee for Space Data Systems, “Recommendation for space data system standards: Turbo codes,” CCSDS 131.2-B-2 Blue Book, 2010.
- [4] C. Berrou, A. Glavieux, and P. Thitimajshima, “Near Shannon limit error-correcting coding and decoding: Turbo-codes,” pp. 1064–1070, 1993.

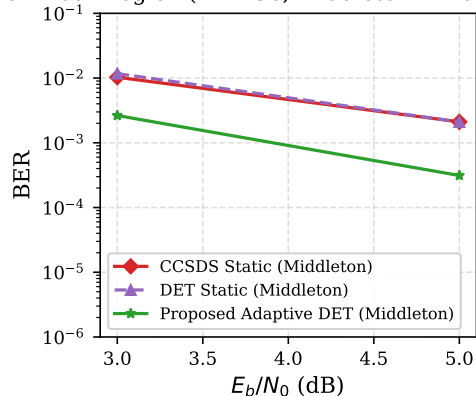
Error Floor Region ($N = 256$, Middleton $A = 0.1$, $\Gamma = 0.01$)

Fig. 7: Zoomed high-SNR region showing error floor of the static CCSDS baseline and its elimination by the proposed adaptive scheme under Middleton Class-A noise ($N = 256$).

- [5] L. R. Bahl, J. Cocke, F. Jelinek, and J. Raviv, "Optimal decoding of linear codes for minimizing symbol error rate," *IEEE Trans. Inf. Theory*, vol. 20, no. 2, pp. 284–287, 1974.
- [6] S. Benedetto and G. Montorsi, "Unveiling turbo codes: Some results on parallel concatenated coding schemes," *IEEE Trans. Inf. Theory*, vol. 42, no. 2, pp. 409–428, 1996.
- [7] S. Dolinar and D. Divsalar, "Weight distributions for turbo codes using random and nonrandom permutations," *JPL TDA Progress Report 42-122*, 1995.
- [8] J. Hagenauer, "Rate-compatible punctured convolutional codes (RCPC codes) and their applications," *IEEE Trans. Commun.*, vol. 36, no. 4, pp. 389–400, 1988.
- [9] S. Benedetto, D. Divsalar, G. Montorsi, and F. Pollara, "Serial concatenation of interleaved codes: performance analysis, design and iterative decoding," *IEEE Trans. Inf. Theory*, vol. 44, no. 3, pp. 909–926, 1998.
- [10] K. Fazel and S. Kaiser, "Adaptive coding and modulation for mobile communications," Wiley, 2003.
- [11] N. Dimopoulos and C. Berrou, "Stopping criteria for turbo codes," *Electronics Letters*, vol. 36, no. 15, pp. 1293–1295, 2000.
- [12] A. Shokrollahi, "LDPC codes: An introduction," *Coding, Cryptography and Combinatorics*, 2004.
- [13] J. G. Proakis and M. Salehi, "Digital communications," McGraw-Hill, 2008.
- [14] W. Xu, H. Wang, and Y. Liang, "Deep learning based turbo decoding under impulsive noise," in *Proc. IEEE WCNC*, 2020.